



CSE370 : Database Systems

Project Report

Project Title : “অনুকূল : সবার জন্যে”

Group No : 15, CSE370 Lab Section : 06, Summer 2025		
ID	Name	Contribution
23101122	YASIR	Frontend+Backend+SQL
22301563	ZUNAYED ISLAM	Backend+SQL
22301659	SAMMAM ZUHAYER	Frontend+SQL

Table of Contents

Section No	Content	Page No
1	Introduction	3
2	Project Features	4-5
3	ER/EER Diagram	6
4	Schema Diagram	6
5	Normalization	6
6	Frontend Development	7-13
7	Backend Development	14-20
8	Source Code Repository	21
9	Conclusion	21-22
10	References	23

Introduction

Our project “Onukul” is designed as a complete digital platform to simplify and modernize humanitarian aid management. It combines 4 essential roles; admin, volunteer, donor, and aid seeker, within a single integrated system, ensuring smooth operations, accountability, and fairness in distribution.

The admin plays the role of the system’s backbone. They are responsible for creating aid packages by grouping essential items such as food, clothing, and medicine. The admin also adds or updates individual items in the database, maintaining accurate inventory control. A critical responsibility is marking the distribution of aid once it is delivered. This prevents duplication, strengthens accountability, and provides a clear record of completed distributions. By defining aid-related data structures in SQL, the admin ensures that every package and item is properly documented, retrievable, and transparent.

Volunteers are the bridge between technology and real world operations. They use the map integration to mark affected areas, input ground level data such as district, union, and city, and update the database accordingly. This location-based feature ensures that no community is left behind and avoids unnecessary overlaps in distribution. Volunteers also mark completed distributions directly on the map, creating a visual record of progress. This mapping function adds clarity and fairness to the entire aid system.

The donor is one of the most crucial contributors to this platform. Donors provide financial resources, which are securely stored in the SQL database for tracking and reporting. This ensures transparency in the use of funds and builds trust in the system. Their donations make it possible to fund aid packages, logistics, and distribution activities.

On the other hand, the aid seeker benefits directly from the platform. Aid seekers can request either specific items (such as rice or medicine) or complete predefined packages designed by the admin. Their requests are recorded in the database, ensuring needs are addressed quickly and systematically. By allowing direct input from seekers, the system remains responsive to real human demands.

Ultimately, “Onukul” is a technology-driven humanitarian support system that connects resources with those in need through an organized, transparent, and fair process. It

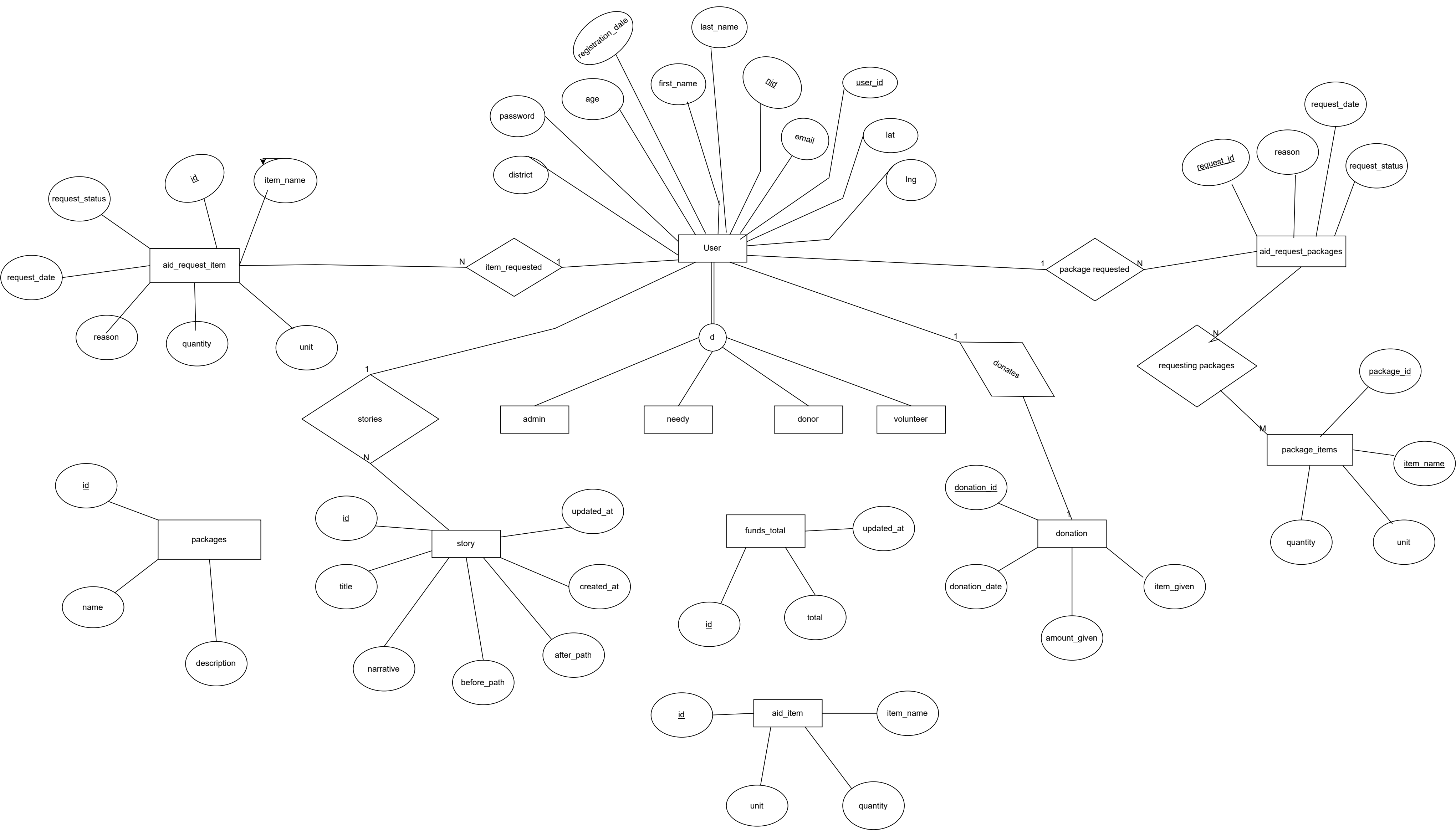
reduces mismanagement, builds trust, ensures accountability, and creates an efficient bridge between donors, volunteers, and seekers strengthening community resilience and support.

Project Features

ID, Name	Features [3 per member]	
23101122, YASIR	Ft 1	Donor can donate money, which will be added at the database; as well as the total funds raised amount will be shown at every user interface's homepage. Also, Donor can donate aid items; which will also be stored in the inventory at database.
	Ft 2	Aid seeker can request for any particular aid item; where in a page, a complete list of inventory will be shown. From there, the aid seeker can choose any item & request for it
	Ft 3	Aid seeker can request for any particular Aid package; where in a page, a complete list of aid packages will be shown. From there, the aid seeker can choose any package & request for it.
22301563 ZUNAYED ISLAM	Ft 1	The admin creates structured aid packages by grouping essential goods like food, medicine, and clothing, ensuring systematic distribution and uniform support for all registered beneficiaries.
	Ft 2	They manage individual aid items, recording them in the database to maintain accurate stock levels, prevent shortages, and ensure proper inventory tracking for effective resource control.

	Ft 3	Admins mark completed distributions and structure related SQL tables, providing accountability, transparent record-keeping, and organized data management that ensures trust and efficiency across the entire aid system.
22301659 SAMMAM ZUHAYER	Ft 1	Volunteers draw maps to identify and highlight affected regions, creating a clear visualization of where help is required, ensuring accurate aid targeting and efficient planning.
	Ft 2	They input detailed local information such as district, union, and city, so that digital mapping reflects ground realities accurately, ensuring precise, location-based aid distribution management.
	Ft 3	Volunteers update maps to mark distributed areas and record this data in SQL, preventing duplication, ensuring equal coverage, and making the system location-aware and reliable.

ER/EER Diagram & Schema Diagram



user

<u>user-id</u>	first-name	last-name	nid	age	type	district	password
----------------	------------	-----------	-----	-----	------	----------	----------

registration-date	email	phone	lat	lng
-------------------	-------	-------	-----	-----

aid-request-item

<u>id</u>	item-name	unit	quantity	reason	request-status	created-at	user-id
-----------	-----------	------	----------	--------	----------------	------------	---------

donation

<u>donation-id</u>	donation-date	amount-given	item-given	user-id
--------------------	---------------	--------------	------------	---------

story

<u>id</u>	title	narrative	before-path	after-path	created-at	updated-at	user-id
-----------	-------	-----------	-------------	------------	------------	------------	---------

aid-request-packages

<u>request-id</u>	reason	request-date	request-status	user-id
-------------------	--------	--------------	----------------	---------

packages-items

<u>package-id</u>	item-name	unit	quantity
-------------------	-----------	------	----------

requested-packages

<u>package-id</u>	<u>request-id</u>
-------------------	-------------------

aid-item

<u>id</u>	item-name	unit	quantity
-----------	-----------	------	----------

funds-total

<u>id</u>	total	updated
-----------	-------	---------

Normalization

- a. Explain if your converted Schema is in 1NF or not. If not, decompose it to 1NF.
- b. Explain if your converted Schema is in 2NF or not. If not, decompose it to 2NF. Can there be any partial functional dependencies in your relational schema?
- c. Explain if your converted Schema is in 3NF or not. If not, decompose it to 3NF. Can there be any transitive dependencies in your relational schema?

Yes, the above Schema diagram is in 3rd normal form.

To be eligible for 3rd normal form, the schema has to be first at 1st normal form, then 2nd normal norm, then if there is no transitive dependencies, then it can be at 3rd normal form.

Since there was no composite values, nested relationships, and multi-valued attributes, the above Schema diagram is in 1st normal form.

And, as there was no partial functional dependencies in the above Schema diagram, so it is also in 2nd normal form.

Also, since there was no any transitive dependencies in the above Schema diagram, so it is in 3rd normal form.

Frontend Development

Contribution of ID : 23101122, Name : YASIR

```
1 window.onDonateMoney = async function(e){
2   e.preventDefault();
3   const amt = Number(new FormData(e.target).get('amount'));
4   try{
5     await donateMoney(amt);
6     toast('Thank you for your donation!', 'success');
7     state.stats = await fetchStats().catch(()=>state.stats);
8     render(); // refresh header funds chip
9   }
10  }catch(err){ toast(err.message|| 'Donation failed', 'error'); }
11 };
12 window.onDonateItem = async function(e){
13   e.preventDefault();
14   const f = new FormData(e.target);
15   const payload = { item_name: f.get('item_name'), unit: f.get('unit'), quantity: Number(f.get('quantity')) };
16   try{
17     await donateItem(payload);
18     toast('Item added to inventory - thank you!', 'success');
19     state.inventory = await fetchInventory().catch(()=>state.inventory||[]);
20   }catch(err){ toast(err.message|| 'Donation failed', 'error'); }
21 };
```



```
1  async function onAidRequest(e){
2    e.preventDefault();
3    const f=new FormData(e.target);
4    const mode = f.get('mode') || 'package';
5    const reason = f.get('reason') || '';
6    try{
7      if (mode === 'package') {
8        const f = new FormData(e.target);
9        const keys = [...f.keys()].filter(k => k.startsWith('pkgqty_'));
10       const picks = keys.map(k => {
11         const id = Number(k.split('_')[1]);
12         const q = Number(f.get(k));
13         return (q > 0) ? { package_id:id, quantity:q } : null;
14       }).filter(Boolean);
15
16       if (picks.length === 0) { throw new Error('Enter quantity for at least one package'); }
17
18       for (const p of picks) {
19         await createAidRequest({ mode:'package', package_id:p.package_id, quantity:p.quantity, reason });
20       }
21
22       toast('Application submitted','success');
23       state.aidRequests = await fetchAidRequests().catch(()=>[]);
24       render();
25     }
26
27     else if (mode === 'item') {
28       const f = new FormData(e.target);
29
30       // gather every row where quantity_# > 0
31       const qtyKeys = [...f.keys()].filter(k => k.startsWith('quantity_'));
32       const items = qtyKeys.map(k => {
33         const idx = k.split('_')[1];
34         const qty = Number(f.get(k));
35         if (qty > 0) {
36           return {
37             item_name: f.get('item_name_' + idx),
38             unit: f.get('unit_' + idx),
39             quantity: qty
40           };
41         }
42         return null;
43       }).filter(Boolean);
44
45       if (items.length === 0) { throw new Error('Enter quantity for at least one item'); }
46
47       // submit one request per filled row (no server change needed)
48       for (const it of items) {
49         await createAidRequest({ mode:'item', item_name:it.item_name, unit:it.unit, quantity:it.quantity, reason });
50       }
51
52       toast('Application submitted','success');
53       state.aidRequests = await fetchAidRequests().catch(()=>[]);
54       render();
55     }
56
57   }catch(err){ toast(err.message||'Submit failed','error'); }
58 }
59
60
```

```

-- Table structure for table `donation`
CREATE TABLE `donation` (
  `donation_id` int(11) NOT NULL,
  `donated_by` int(11) DEFAULT NULL,
  `donation_date` date NOT NULL,
  `amount_given` decimal(12,2) DEFAULT NULL,
  `item_given` varchar(120) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Table structure for table `funds_total`
CREATE TABLE `funds_total` (
  `id` tinyint(4) NOT NULL DEFAULT 1,
  `total` decimal(14,2) NOT NULL DEFAULT 0.00,
  `updated_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Table structure for table `aid_request`
CREATE TABLE `aid_request` (
  `request_id` int(11) NOT NULL,
  `user_id` int(11) NOT NULL,
  `package_id` int(11) DEFAULT NULL,
  `reason` text DEFAULT NULL,
  `request_date` date NOT NULL DEFAULT curdate(),
  `request_status` enum('pending','approved','rejected') NOT NULL DEFAULT 'pending'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Table structure for table `aid_request_item`
CREATE TABLE `aid_request_item` (
  `id` int(11) NOT NULL,
  `request_id` int(11) NOT NULL,
  `item_name` varchar(120) NOT NULL,
  `unit` varchar(40) NOT NULL,
  `quantity` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

Contribution of ID : 22301563, Name : ZUNAYED RAHMAN



```
1 function InventoryAdmin(){ return Packages(); }
2
3 window.onAddInventory = async function(e){
4   e.preventDefault();
5   const f=new FormData(e.target);
6   await addInventory(f.get('item_name'), f.get('unit'), Number(f.get('quantity')));
7   toast('Inventory updated','success');
8   state.inventory = await fetchInventory();
9   render();
10 };
11 window.onEditInventory = async function(item_name, unit, currentQty){
12   const newQty = Number(prompt(`Update quantity for ${item_name} (${unit})`, currentQty));
13   if(!Number.isFinite(newQty)) return;
14   await jsonFetch('api.php?action=inventory', {
15     method:'POST', headers: {'Content-Type':'application/json'},
16     body: JSON.stringify({ op:'update', item_name, unit, quantity:newQty })
17   });
18   toast('Inventory updated','success');
19   state.inventory = await fetchInventory();
20   render();
21 };
```



```
1 window.onAddPackage = async function(e){
2   e.preventDefault();
3   const f=new FormData(e.target);
4   await jsonFetch('api.php?action=packages', {
5     method:'POST', headers: {'Content-Type':'application/json'},
6     body: JSON.stringify({ op:'add_package', name: f.get('name'), description: f.get('description') })
7   });
8   toast('Package created','success');
9   const pk = await fetchPackages();
10  state.packages = joinPackages(pk);
11  render();
12 };
13
14 window.onAddPackageItem = async function(e){
15   e.preventDefault();
16   const f=new FormData(e.target);
17   await jsonFetch('api.php?action=packages', {
18     method:'POST', headers: {'Content-Type':'application/json'},
19     body: JSON.stringify({
20       op:'add_item',
21       package_id: Number(f.get('package_id')),
22       item_name: f.get('item_name'),
23       unit: f.get('unit'),
24       quantity: Number(f.get('quantity'))
25     })
26   });
27   toast('Item added to package','success');
28   const pk = await fetchPackages();
29   state.packages = joinPackages(pk);
30   render();
31 };
```

```

-- Table structure for table `packages`
CREATE TABLE `packages` (
  `id` int(11) NOT NULL,
  `name` varchar(120) NOT NULL,
  `description` text DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Table structure for table `package_items`
CREATE TABLE `package_items` (
  `id` int(11) NOT NULL,
  `package_id` int(11) NOT NULL,
  `item_name` varchar(120) NOT NULL,
  `quantity` int(11) NOT NULL,
  `unit` varchar(40) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Sample data for table `packages`
INSERT INTO `packages` (`id`, `name`, `description`) VALUES
(1, 'Basic Food Pack', 'Rice, lentils, oil, salt'),
(2, 'Hygiene Pack', 'Soap, sanitizer, masks'),
(3, 'Water Relief', 'Drinking water bottles');

-- Sample data for table `package_items`
INSERT INTO `package_items` (`id`, `package_id`, `item_name`, `quantity`, `unit`) VALUES
(5, 1, 'Rice', 5, 'kg'),
(6, 1, 'Lentils', 2, 'kg'),
(7, 1, 'Oil', 1, 'L'),
(8, 1, 'Salt', 1, 'kg'),
(9, 2, 'Soap', 4, 'bar'),
(10, 2, 'Sanitizer', 2, 'bottle'),
(11, 2, 'Masks', 10, 'pcs'),
(12, 3, 'Water', 12, 'bottle');

```

Contribution of ID : 22301659, Name : SAMMAM ZUHAYER



```
1 window.onDistributed = async function(id){
2   try{
3     await jsonFetch('api.php?action=aidRequests', {
4       method:'POST', headers:{'Content-Type':'application/json'},
5       body: JSON.stringify({ op:'distribute', request_id: id })
6     });
7     toast('Marked as distributed','success');
8     state.aidRequests = await jsonFetch('api.php?action=aidRequests');
9     render();
10  }catch(e){ toast(e.message||'Failed','error'); }
11 };
12
```



```
1 // ----- volunteer map init -----
2
3 let __leaflet, __markerLayer;
4 async function initVolunteerMap(){
5
6   const rows = await getLocations().catch(()=>[]);
7
8   // always recreate the map when (re)entering the page
9   if (__leaflet) { try { __leaflet.remove(); } catch(e) {} __leaflet = null; }
10  __leaflet = L.map('vol-map').setView([23.777,90.399],7);
11  L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png',
12    {maxZoom:19, attribution:'&copy; OpenStreetMap'}).addTo(__leaflet);
13  setTimeout(()=>__leaflet.invalidateSize(), 100);
14
15  if(__markerLayer){ try{ __markerLayer.remove(); }catch(e){} }
16  __markerLayer = L.layerGroup().addTo(__leaflet);
17
18
19
20  rows.forEach(v=>{
21    const lat=parseFloat(v.lat), lng=parseFloat(v.lng);
22    if(!isNaN(lat)&&!isNaN(lng)){
23      L.marker([lat,lng]).addTo(__markerLayer)
24      .bindPopup(`<strong>${v.name}||'Volunteer'</strong><br>${v.district}||'`);
25    }
26  });

```



```
1 window.onSetLocationDistrict = async function(){
2   const district = document.querySelector('select[name="district"]')?.value;
3   if (!district) { toast('Please choose a district','error'); return; }
4   const c = DIST_CENTROID[district];
5   const lat = c ? c[0] : null, lng = c ? c[1] : null;
6   try{
7     await setMyLocation({ lat, lng, district });
8     toast('Location updated','success');
9     await initVolunteerMap();
10  }catch(e){ toast(e.message||'Update failed','error'); }
11  };
12
```

Backend Development

Contribution of ID : 23101122, Name : YASIR

```
1 // ---- donations / stats ----
2 case 'donations': {
3     $pdo = pdo();
4     if ($_SERVER['REQUEST_METHOD'] === 'POST') {
5         $b = jbody();
6         $mode = $b['mode'] ?? '';
7         $who = uid(); // donors/admins ideally
8         if ($mode === 'money') {
9             $amt = (float)($b['amount'] ?? 0);
10            if ($amt <= 0) fail('Amount must be positive');
11            $st = $pdo->prepare("INSERT INTO donation (donated_by, donation_date, amount_given) VALUES (?, CURDATE(), ?)");
12            $st->execute([$who, $amt]);
13            // bump funds_total
14            $pdo->exec("UPDATE funds_total SET total = total + ".(float)$amt." WHERE id=1");
15            ok(['ok'=>true]);
16        } elseif ($mode === 'item') {
17            $name = trim($b['item_name'] ?? '');
18            $unit = trim($b['unit'] ?? '');
19            $qty = (int)($b['quantity'] ?? 0);
20            if ($name==='' || $unit==='' || $qty<=0) fail('Invalid item donation');
21            // upsert inventory
22            $st = $pdo->prepare("SELECT id FROM aid_item WHERE item_name=? AND unit=? LIMIT 1");
23            $st->execute([$name,$unit]);
24            if ($row = $st->fetch()) {
25                $st = $pdo->prepare("UPDATE aid_item SET quantity=quantity+? WHERE id=?");
26                $st->execute([$qty, (int)$row['id']]);
27            } else {
28                $st = $pdo->prepare("INSERT INTO aid_item (item_name,unit,quantity) VALUES (?,?,:)");
29                $st->execute([$name,$unit,$qty]);
30            }
31            // log donation
32            $item_given = $qty.' '.$unit.' '.$name;
33            $st = $pdo->prepare("INSERT INTO donation (donated_by, donation_date, item_given) VALUES (?, CURDATE(), ?)");
34            $st->execute([$who, $item_given]);
35            ok(['ok'=>true]);
36        } else {
37            fail('Invalid donation mode');
38        }
39    } else {
40        $rows = $pdo->query("SELECT * FROM donation ORDER BY donation_id DESC")->fetchAll();
41        ok($rows);
42    }
43 }
```




```
1 // ---- aid requests (create/list/approve/reject/distribute) ----
2 case 'aidRequests': {
3     $pdo = pdo();
4     if ($_SERVER['REQUEST_METHOD'] === 'POST') {
5         $b = jbody();
6         $op = strtolower($b['op'] ?? 'create'); // default to create
7
8         if ($op === 'create' || $op === 'request') {
9             $user_id = user_or_401();
10            $mode = $b['mode'] ?? 'package';
11
12            if ($mode === 'package') {
13                $package_id = (int)($b['package_id'] ?? 0);
14                $reason = trim($b['reason'] ?? '');
15                if ($package_id <= 0) fail('package_id required');
16                // NOTE: table has no quantity column; we store the request without qty to match schema
17                $st = $pdo->prepare("INSERT INTO aid_request_packages (user_id, package_id, reason) VALUES (?, ?, ?)");
18                $st->execute([$user_id, $package_id, $reason]);
19                ok(['request_id' => (int)$pdo->lastInsertId(), 'mode' => 'package']);
20            } elseif ($mode === 'item') {
21                $item = trim($b['item_name'] ?? '');
22                $unit = trim($b['unit'] ?? '');
23                $qty = (int)($b['quantity'] ?? 0);
24                $reason = trim($b['reason'] ?? '');
25                if ($item === '' || $unit === '' || $qty <= 0) fail('item_name, unit, quantity required');
26                $st = $pdo->prepare("INSERT INTO aid_request_item (user_id, item_name, unit, quantity, reason) VALUES (?, ?, ?, ?, ?)");
27                $st->execute([$user_id, $item, $unit, $qty, $reason]);
28                ok(['request_id' => (int)$pdo->lastInsertId(), 'mode' => 'item']);
29            } else {
30                fail('Invalid mode');
31            }
32        }
33    }
```

```

1 // item requests
2 $sql1 = "
3     SELECT r.id AS request_id, 'item' AS mode, r.reason, r.request_status,
4           u.first_name, u.last_name, NULL AS package_name
5     FROM aid_request_item r
6    JOIN user u ON u.user_id = r.user_id
7    " . ($whereMine ? "WHERE r.id = " . $id : "") . "
8    ORDER BY r.id DESC
9 ";
10 $st1 = $pdo->prepare($sql1);
11 $st1->execute($params);
12 $items = $st1->fetchAll();
13
14 // fetch item rows detail
15 $itemRows = [];
16 if ($items) {
17     $ids = array_column($items, 'request_id');
18     // each item request is a single item row (schema has one row per item request)
19     $in = implode(',', array_fill(0, count($ids), '?'));
20     $rs = $pdo->prepare("SELECT id AS request_id, item_name, unit, quantity FROM aid_request_item WHERE id IN ($in)");
21     $rs->execute($ids);
22     $by = [];
23     foreach ($rs as $r) { $by[(int)$r['request_id']] = ['item_name'=>$r['item_name'], 'unit'=>$r['unit'], 'quantity'=>(int)$r['quantity']]; }
24     $itemRows = $by;
25 }
26
27 foreach ($items as $r) {
28     $r['name'] = trim(($r['first_name'] ?? '') . ' ' . ($r['last_name'] ?? ''));
29     $r['items'] = $itemRows[(int)$r['request_id']] ?? [];
30     unset($r['first_name'], $r['last_name']);
31 }
32
33 // package requests
34 $sql2 = "
35     SELECT r.request_id, 'package' AS mode, r.reason, r.request_status,
36           u.first_name, u.last_name, p.name AS package_name, p.id AS package_id
37     FROM aid_request_packages r
38    JOIN user u ON u.user_id = r.user_id
39    JOIN packages p ON p.id = r.package_id
40    " . ($whereMine ? "WHERE r.id = " . $id : "") . "
41    ORDER BY r.request_id DESC
42 ";
43 $st2 = $pdo->prepare($sql2);
44 $st2->execute($params);
45 $packs = $st2->fetchAll();
46
47 // attach package items
48 $byPkg = [];
49 if ($packs) {
50     $pids = array_values(array_unique(array_column($packs, 'package_id')));
51     $in = implode(',', array_fill(0, count($pids), '?'));
52     $rs = $pdo->prepare("SELECT package_id, item_name, unit, quantity FROM package_items WHERE package_id IN ($in)");
53     $rs->execute($pids);
54     foreach ($rs as $r) {
55         $byPkg[(int)$r['package_id']] = ['item_name'=>$r['item_name'], 'unit'=>$r['unit'], 'quantity'=>(int)$r['quantity']];
56     }
57 }
58 foreach ($packs as $r) {
59     $r['name'] = trim(($r['first_name'] ?? '') . ' ' . ($r['last_name'] ?? ''));
60     $r['items'] = $byPkg[(int)$r['package_id']] ?? [];
61     unset($r['first_name'], $r['last_name'], $r['package_id']);
62 }
63
64 ok(array_merge($packs, $items));
65 }

```

Contribution of ID : 22301563, Name : ZUNAYED RAHMAN

```

1 // Admin/Volunteer ops
2 if (in_array($op, ['approve', 'reject', 'distribute'], true)) {
3     $rid = (int)($b['request_id'] ?? 0);
4     if ($rid <= 0) fail('request_id required');
5
6     // Try item table first
7     $st = $pdo->prepare("SELECT id, request_status FROM aid_request_item WHERE id=?");
8     $st->execute([$rid]);
9     if ($row = $st->fetch()) {
10         $new = ($op === 'approve') ? 'approved' : (($op === 'reject') ? 'rejected' : 'distributed');
11         $st = $pdo->prepare("UPDATE aid_request_item SET request_status=? WHERE id=?");
12         $st->execute([$new, $rid]);
13         ok(['ok'=>true, 'mode'=>'item', 'status'=>$new]);
14     }
15
16     // Then package table (enum lacks 'distributed'; map to approved on distribute)
17     $st = $pdo->prepare("SELECT request_id, request_status FROM aid_request_packages WHERE request_id=?");
18     $st->execute([$rid]);
19     if ($row = $st->fetch()) {
20         $new = ($op === 'approve') ? 'approved' : (($op === 'reject') ? 'rejected' : 'approved');
21         $st = $pdo->prepare("UPDATE aid_request_packages SET request_status=? WHERE request_id=?");
22         $st->execute([$new, $rid]);
23         ok(['ok'=>true, 'mode'=>'package', 'status'=>$new]);
24     }
25
26     fail('Request not found', 404);
27 }
28
29 fail('Invalid action', 400);
30 }
31
32 // GET: list combined queue (admin/volunteer see all; needy sees own)
33 $u = uid();
34 if (!$u) ok([]); // not logged in -> empty
35
36 $whereMine = '';
37 $params = [];
38 // If the user is needy, show own requests only
39 $st = $pdo->prepare("SELECT type FROM user WHERE user_id=?");
40 $st->execute([$u]);
41 $role = $st->fetchColumn();
42 if ($role === 'needy') { $whereMine = ' WHERE r.user_id=? '; $params[] = $u; }
43
44 $pdo = pdo();

```



```
1  // ---- inventory ----
2  case 'inventory': {
3      $pdo = pdo();
4      if ($_SERVER['REQUEST_METHOD'] === 'POST') {
5          $b = jbody();
6          $op = strtolower($b['op'] ?? '');
7          $item = trim($b['item_name'] ?? '');
8          $unit = trim($b['unit'] ?? '');
9          $qty = (int)($b['quantity'] ?? 0);
10         if ($item === '' || $unit === '' || $qty < 0) fail('Invalid inventory payload');
11
12         // find by item+unit
13         $st = $pdo->prepare("SELECT id, quantity FROM aid_item WHERE item_name=? AND unit=? LIMIT 1");
14         $st->execute([$item,$unit]);
15         $row = $st->fetch();
16
17         if ($op === 'update') {
18             if (!$row) fail('Item not found for update');
19             $st = $pdo->prepare("UPDATE aid_item SET quantity=? WHERE id=?");
20             $st->execute([$qty, (int)$row['id']]);
21         } else {
22             if ($row) {
23                 $st = $pdo->prepare("UPDATE aid_item SET quantity=quantity+? WHERE id=?");
24                 $st->execute([$qty, (int)$row['id']]);
25             } else {
26                 $st = $pdo->prepare("INSERT INTO aid_item (item_name,unit,quantity) VALUES (?,?,?)");
27                 $st->execute([$item,$unit,$qty]);
28             }
29         }
30         ok(['ok'=>true]);
31     } else {
32         $rows = $pdo->query("SELECT item_name, unit, quantity FROM aid_item ORDER BY item_name")->fetchAll();
33         ok($rows);
34     }
35 }
```

```

1  // ---- packages ----
2  case 'packages': {
3      $pdo = pdo();
4      if ($_SERVER['REQUEST_METHOD'] === 'POST') {
5          $b = jbody();
6          $op = strtolower($b['op'] ?? '');
7          if ($op === 'add_package') {
8              $name = trim($b['name'] ?? '');
9              if ($name === '') fail('Package name required');
10             $st = $pdo->prepare("INSERT INTO packages (name, description) VALUES (?,?)");
11             $st->execute([$name, $b['description'] ?? null]);
12             ok(['id' => (int)$pdo->lastInsertId()]);
13         } elseif ($op === 'add_item') {
14             $pid = (int)($b['package_id'] ?? 0);
15             $iname = trim($b['item_name'] ?? '');
16             $unit = trim($b['unit'] ?? '');
17             $qty = (int)($b['quantity'] ?? 0);
18             if ($pid<=0 || $iname==='' || $unit==='' || $qty<=0) fail('Invalid item payload');
19             // Upsert into package_items (PK: package_id,item_name,unit)
20             $st = $pdo->prepare("
21                 INSERT INTO package_items (package_id,item_name,unit,quantity)
22                 VALUES (?,?,?,?)
23                 ON DUPLICATE KEY UPDATE quantity=VALUES(quantity)
24             ");
25             $st->execute([$pid,$iname,$unit,$qty]);
26             ok(['ok'=>true]);
27         } else {
28             fail('Invalid action', 400);
29         }
30     } else {
31         // return packages with nested items (what frontend expects)
32         $pk = $pdo->query("SELECT id, name, description FROM packages ORDER BY id")->fetchAll();
33         $it = $pdo->query("SELECT package_id, item_name, unit, quantity FROM package_items")->fetchAll();
34         $by = [];
35         foreach ($it as $r) {
36             $by[(int)$r['package_id']][] = [
37                 'name' => $r['item_name'],
38                 'unit' => $r['unit'],
39                 'quantity' => (int)$r['quantity'],
40             ];
41         }
42         foreach ($pk as $p) {
43             $p['items'] = $by[(int)$p['id']] ?? [];
44         }
45         ok($pk);
46     }
47 }

```

Contribution of ID : 22301659, Name : SAMMAM ZUHAYER

```

1 // ---- volunteer locations ----
2 case 'locations': {
3     $pdo = pdo();
4     if ($_SERVER['REQUEST_METHOD'] === 'POST') {
5         $u = user_or_401();
6         $b = json_body();
7         $lat = isset($b['lat']) ? (float)$b['lat'] : null;
8         $lng = isset($b['lng']) ? (float)$b['lng'] : null;
9         $district = $b['district'] ?? null;
10        $st = $pdo->prepare("UPDATE user SET lat=?, lng=?, district=COALESCE(?, district) WHERE user_id=?");
11        $st->execute([$lat, $lng, $district, $u]);
12        ok(['ok'=>true]);
13    } else {
14        $rows = $pdo->query("SELECT user_id, CONCAT(first_name, ' ', last_name) AS name, district, lat, lng FROM user WHERE type IN ('volunteer', 'admin') AND lat IS NOT NULL AND lng IS NOT NULL")->fetchAll();
15        ok($rows);
16    }
17 }

```

— Table structure for table `user` (ensure these columns exist)

```

CREATE TABLE `user` (
  `user_id` int(11) NOT NULL,
  `first_name` varchar(100) NOT NULL,
  `last_name` varchar(100) NOT NULL,
  `nid` varchar(100) NOT NULL,
  `age` int(11) NOT NULL,
  `type` enum('needy','volunteer','donor','admin') NOT NULL,
  `district` varchar(120) DEFAULT NULL,
  `password` varchar(255) NOT NULL,
  `registration_date` date NOT NULL DEFAULT curdate(),
  `email` varchar(255) NOT NULL,
  `phone` varchar(40) NOT NULL,
  `lat` decimal(9,6) DEFAULT NULL,
  `lng` decimal(9,6) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

— Sample volunteer data with map coordinates

```

INSERT INTO `user` (`user_id`, `first_name`, `last_name`, `nid`, `age`, `type`, `district`, `password`, `registration_date`, `email`, `phone`, `lat`, `lng`) VALUES
(8, 'user', 'volunteer 3', '444555666', 34, 'volunteer', 'Gopalganj', '$2y$10$9vuL.R2fe7PG1dvh0ZaerpxUHscFMUyMHpx2EKyEfKZeQYYMKiC', '2025-09-10', 'mohammadrobby11736@gmail.com', '41545484', 23.005000, 89.826000);

```

Source Code Repository

https://github.com/foundyasir/PROJECT_ONUKUL

Conclusion

“Onukul” represents a comprehensive digital solution for humanitarian aid distribution, successfully bridging the gap between those in need and those willing to help. This web application effectively demonstrates how technology can be leveraged to create efficient, transparent, and accessible systems for social welfare.

Key Achievements:

For Administrators: The platform provides robust tools for managing aid inventory, creating customized assistance packages, and overseeing the entire distribution process with real-time tracking capabilities.

For Volunteers: The interactive mapping system enables efficient coordination and geographical management of aid distribution, ensuring resources reach the right locations at the right time.

For Donors: The system offers seamless donation processes for both monetary contributions and physical goods, with transparent tracking of how contributions are utilized.

For Aid Seekers: The intuitive interface allows those in need to request specific items or pre-configured aid packages based on their unique circumstances and requirements.

Technical Excellence:

Built with modern web technologies including PHP, MySQL, JavaScript, and Leaflet.js for mapping, Onukul demonstrates a scalable architecture that can handle multiple

user roles, real-time inventory management, and geographical data visualization. The responsive design ensures accessibility across various devices, making aid management possible even in resource-constrained environments.

Social Impact:

By digitizing the aid distribution process, Onukul reduces administrative overhead, minimizes resource waste, and ensures help reaches those who need it most. The platform's transparency builds trust among all stakeholders—donors can see the impact of their contributions, administrators can optimize resource allocation, and recipients can track the status of their requests.

Future Potential:

The foundation established by Onukul opens possibilities for expansion including mobile applications, integration with financial services for direct cash transfers, predictive analytics for disaster response, and blockchain technology for enhanced transparency in the aid distribution chain.

“Onukul” stands as a testament to how technology can be harnessed for social good, creating a more efficient and compassionate system for supporting vulnerable communities during times of need.

References

<https://www.youtube.com/playlist?list=PLq1kE3IvmqQ6ek6zHK0UfUvgMGqLGyqbH>

<https://www.youtube.com/playlist?list=PLnGVQFIQATU0tnVup6ky45DWmDGA1oN0T>

<https://www.youtube.com/playlist?list=PLa4DMmNZGejluPe9POvs6dzTRK6Ho6FRF>

https://www.youtube.com/playlist?list=PLbrkVe8jT_YwMhF4TWw1Q9ETFrUIYjOPM

<https://www.youtube.com/playlist?list=PLuW5vAehjsxo37-OOyChO9yDYwZKcBLZ9>